

TÀI LIỆU HỌC & BÁO CÁO — 10 KỊCH BẢN TẤN CÔNG ↔ PHÒNG THỦ

File học + báo cáo cho cả 3 thành viên. Mỗi kịch bản gồm (A) **Hướng tấn công** (cách đánh + vì sao có lỗ hổng + payload thật, chỉ rõ ở dòng nào trong `scripts/attack_demo.sh` và giải thích payload chạy ra sao) và (B) **Hướng phòng thủ** (mindset + code chặn, ghi theo cú pháp `file:line` + giảng kỹ luồng xử lý của đoạn code đó).

Phân công 3 người (công sức cân bằng)

Người	Mảng	Kịch bản	SL	Vì sao cân bằng
Người 1	JWT / Định danh	KB1, KB6, KB9	3	Đào sâu code mật mã (<code>jwt_verifier.py</code> , <code>auth.py</code>)
Người 2	HMAC / M2M	KB2, KB3, KB5	3	Đào sâu code (<code>hmac_verifier.py</code> , <code>audit.py</code> , <code>rate_limit.py</code>)
Người 3	Hạ tầng / Truyền dẫn	KB4, KB7, KB8, KB10	4	Mỗi cái nhẹ code hơn (control triển khai) → $4 \approx 3$

Nền tảng chung — đọc trước (cả 3 người)

Dây chuyền chốt — request đi qua đâu

Một request vào Gateway đi qua các **middleware** rồi tới **endpoint**. Hai middleware xác thực, mỗi cái chỉ gác đúng đường của nó:

- `gateway/middleware/auth.py:19` — gác `/api/protected`, `/api/admin` (luồng **JWT/user**).
- `gateway/middleware/hmac_auth.py:25` — gác `/api/service` (luồng **HMAC/M2M**).
- `gateway/main.py:65-86` — các endpoint, mỗi endpoint có `@limiter.limit(...)` (rate-limit).

Thứ tự kiểm tra từng luồng theo nguyên tắc **rẻ** → **đắt** → **quyền**:

- [1] rate-limit (@limiter) quá ngưỡng → 429 (rẻ nhất)
- [2] có token / đúng định dạng? thiếu/méo → 401
- [3] exp / timestamp hết/cũ → 401
- [4] nonce trùng → 401 (chống replay)
- [5] HMAC body sai → 401 (chống sửa body)
- [6] verify chữ ký JWT sai → 401 ← AUTHENTICATION ("anh là ai")
- [7] scope / role thiếu → 403 ← AUTHORIZATION ("được làm gì")
- [8] backend tự kiểm lại (zero-trust)

Hai quy tắc vàng (lặp lại suốt file)

1. **401 ≠ 403**. 401 = chưa chứng minh được danh tính (authentication). 403 = biết danh tính rồi nhưng **thiếu quyền** (authorization). Không trộn.
2. **Rẻ trước, đắt sau**. Kiểm rẻ (format/timestamp/nonce) đặt TRƯỚC kiểm tốn CPU (verify chữ ký) → rác bị loại sớm, chống DoS tăng ứng dụng.

Chạy thử toàn bộ

```
bash scripts/attack_demo.sh    # bắn live, mỗi cảnh = 1-vài kịch bản, in PASS/FAIL
```

NGƯỜI 1 — JWT / ĐỊNH DANH

KB1 — Giả mạo token (Spoofing · MT-AUTHN)

(A) Hướng tấn công

Cách đánh & vì sao có lỗ hổng. JWT gồm 3 phần header.payload.signature. Kẻ tấn công muốn được Gateway coi là user hợp lệ, nên nó tự dựng token. Có 2 đường:

- **Đường 1 — ký bằng khóa lạ:** nó điền sub, roles tùy ý rồi ký bằng **khóa của nó** (vì không có khóa thật của IdP). Lỗ hổng xảy ra **nếu** Gateway verify bằng khóa do chính token gợi ý, hoặc không verify chữ ký.
- **Đường 2 — hạ cấp alg=none:** JWT cho phép trường alg trong header khai báo thuật toán. Nếu verifier **tin** trường này, kẻ tấn công đặt alg=none nghĩa là "token không cần chữ ký", phần signature để trống → lừa verifier bỏ qua bước kiểm.

Luồng xử lý của đòn đánh: client gửi Authorization: Bearer <token giả> → auth.py:19 bắt đường /api/protected → auth.py:28 gọi verify_token → nếu verifier yếu thì token giả lọt; nếu chắc thì rớt ở bước verify.

Payload (theo scripts/attack_demo.sh:63-66):

```
# Đường 1 — token thật bị đổi 3 ký tự cuối => chữ ký hỏng (mô phỏng "ký sai"):
FORGED="${TOKEN:0:${#TOKEN}-3}xyz"
curl ... /api/protected -H "Authorization: Bearer $FORGED"    # dòng 63-64

# Đường 2 — token alg=none, signature RÕNG, tự ghép từ base64url:
NONE_TOK="$(b64url '{"alg":"none","typ":"JWT"}').$(b64url '{"sub":"attacker",...}')."
curl ... /api/protected -H "Authorization: Bearer $NONE_TOK"    # dòng 65-66
```

Giải thích payload:

- Dòng 63: `${TOKEN:0:${#TOKEN}-3}xyz` lấy token thật, cắt 3 ký tự cuối (nằm trong phần *signature*) rồi nối xyz → chữ ký không còn khớp nội dung. Đây là cách rẻ nhất để tạo "token chữ ký sai" mà không cần khóa.
- Dòng 65: hàm `b64url (attack_demo.sh:25)` mã hóa base64url 2 đoạn JSON header+payload, rồi nối lại với **dấu chấm cuối nhưng không có signature** → đúng dạng token `alg=none`.

(B) Hướng phòng thủ

Mindset: *không bao giờ tin metadata do token tự khai*. Khóa verify do **server chọn** từ nguồn tin cậy; thuật toán phải nằm trong **danh sách trắng**; chữ ký verify bằng **toán**.

Code chặn — `gateway/crypto/jwt_verifier.py`:

1. **Whitelist thuật toán** — `jwt_verifier.py:56-57`:

```
if alg not in ("HS256", "RS256", "ES256"):
    raise TokenInvalid(f"... Algorithm '{alg}' is blacklisted")
```

Verifier đọc alg từ header (`jwt_verifier.py:53`) nhưng **chỉ để đối chiếu danh sách trắng**. `none` không thuộc `{HS256, RS256, ES256}` → ném `TokenInvalid` **ngay**, chưa cần đụng tới chữ ký. → Đường 2 trả 401.

2. **Bắt buộc verify chữ ký bằng khóa THẬT** — `jwt_verifier.py:60-66` bật cờ, và `jwt_verifier.py:91-106` lấy khóa từ JWKS theo kid rồi decode:

```
decode_options = {"verify_signature": True, ...}    # dòng 60-66
key = next((k for k in jwks["keys"] if k["kid"] == kid), None) # dòng 95
jwt.decode(token, key, algorithms=[alg], audience=AUDIENCE, issuer=ISSUER, options=...) # 99-106
```

`jwt.decode` **tính lại** chữ ký bằng khóa công khai thật của IdP. Token ký bằng khóa lạ (hoặc bị đổi 3 ký tự) → chữ ký không khớp → thư viện ném `JWTErrors` → bắt ở `jwt_verifier.py:108-110` đổi thành `TokenInvalid` → `middleware auth.py:29-33` trả **401** **invalid_token**. → Đường 1 trả 401.

Vì sao an toàn về bản chất: chọn **chữ ký bất đối xứng** (ES256) — IdP giữ khóa *riêng* để ký, Gateway chỉ giữ khóa *công khai* để verify (`jwt_verifier.py:90-106`). Khóa ở Gateway có lộ cũng **không giả được token**. (Đây là câu trả lời "vì sao ES256 hơn HS256".)

KB6 — Leo thang quyền (Elevation of Privilege · MT-AUTHZ)

(A) Hướng tấn công

Cách đánh & vì sao có lỗ hổng. Kẻ tấn công đăng nhập thật, có **token hợp lệ hoàn toàn** của một user thường, rồi gọi endpoint **chỉ dành cho admin** (/api/admin). Lỗ hổng xảy ra **nếu** hệ thống chỉ kiểm "token hợp lệ không" (authentication) mà **quên kiểm quyền** (authorization) — khi đó mọi user đăng nhập đều vào được khu admin.

Luồng xử lý của đòn đánh: token này **đúng chữ ký** nên vượt qua chốt [6] (authentication PASS). Điểm sống còn là chốt [7]: hệ thống có kiểm role không.

Payload (theo scripts/attack_demo.sh:89-90):

```
# TOKEN là token THẬT lấy ở dòng 54-56 (client_credentials), KHÔNG có role admin:
curl ... /api/admin -H "Authorization: Bearer $TOKEN"      # dòng 90 -> kỳ vọng 403
```

Giải thích payload: không cần "ché" token — dùng **chính token hợp lệ** đã lấy ở Cảnh 1. Mấu chốt: token này không chứa role admin. Đòn đánh = đem token quyền thấp gõ cửa quyền cao.

(B) Hướng phòng thủ

Mindset: *authentication xong CHƯA đủ — phải hỏi tiếp "được làm gì".* Và phân biệt rạch ròi 401 (chưa biết là ai) với 403 (biết rồi nhưng thiếu quyền).

Code chặn — gateway/middleware/auth.py:50-64 (chốt 7 AUTHORIZATION):

```
REQUIRED_ROLES = {ADMIN_PREFIX: "admin"}      # dòng 15
...
for prefix, required_role in REQUIRED_ROLES.items():      # dòng 50
    if request.url.path.startswith(prefix):      # dòng 51
        roles = payload.get("realm_access", {}).get("roles", []) # dòng 52
        if required_role not in roles:          # dòng 53
            record_failure("jwt", "insufficient_role")
            return JsonResponse(status_code=403,      # dòng 61-63
                               content={"detail": f"forbidden: requires role '{required_role}'"})
```

Giải kỹ luồng: code này chỉ chạy **sau khi** authentication PASS (đã qua auth.py:28 verify_token + auth.py:37 không bị revoke). Tại auth.py:52 nó đọc danh sách role **từ claim realm_access.roles trong token ĐÃ KÝ** — không đọc từ header hay input client. Nếu đường là /api/admin mà role thiếu admin → auth.py:61-63 trả **403** (không phải 401). Vì role nằm trong token đã ký, client **không thể tự thêm**: sửa token là hỏng chữ ký → rớt ở KB1.

Điểm nhấn báo cáo: cùng một token, gọi /api/protected → 200; gọi /api/admin → 403. Chứng minh tầng authz độc lập với authn.

KB9 — Đánh tráo JWKS (Spoofing IdP · S1-IdP)

(A) Hướng tấn công

Cách đánh & vì sao có lỗ hổng. Gateway không giữ sẵn khóa công khai; nó **tải về** từ IdP qua URL JWKS — `jwt_verifier.py:7` (JWKS_URL) và `jwt_verifier.py:20-31` (`_get_jwks`). Nếu kênh tải đó **không có TLS** (hoặc DNS bị đầu độc), kẻ tấn công đứng giữa (MITM) trả về **JWKS giả** chứa public key **của nó**, rồi ký token bằng private key tương ứng, đặt `iss/aud/kid` khớp. Lỗ hổng nằm ở chỗ Gateway **tin tuyệt đối** nội dung tải về.

Luồng xử lý của đòn đánh:

```
Gateway --(HTTP, không TLS)--> "IdP"    ← kẻ MITM chen vào đây
▲                                     trả JWKS GIẢ (key của hacker)
└── verify token bằng key giả → khớp → CHẤP NHẬN
```

Payload (theo demo `tests/security/sec_jwks_substitution_demo.py`):

```
jv._get_jwks = lambda: {"keys": [attacker_jwk]}      # mô phỏng kênh JWKS bị tráo
forged = jwt.encode({"sub": "attacker", "iss": "ISSUER", "aud": "AUDIENCE",
                    "realm_access": {"roles": ["admin"]}, ...},
                    attacker_private_pem, algorithm="ES256",
                    headers={"kid": "attacker-kid-001"}) # hacker tự ký + tự phong admin
```

Giải thích payload: dòng `jv._get_jwks = lambda...` thay nguồn khóa bằng **key của hacker** (chính là việc MITM làm ngoài thực tế). Sau đó hacker ký token bằng private key của nó — và vì JWKS đã chứa public key tương ứng, mọi kiểm tra chữ ký đều khớp.

(B) Hướng phòng thủ

Mindset quan trọng nhất kịch bản này: có những đòn tấn công ứng dụng **KHÔNG** tự cứu được — *control phải nằm ở tầng truyền dẫn*. Mọi kiểm tra app-layer (whitelist alg `jwt_verifier.py:56`, verify `iss/aud` `jwt_verifier.py:63-64`, tra `kid` `jwt_verifier.py:95`) **đều PASS** vì token khớp bộ khóa đã bị tráo — hacker kiểm soát cả token lẫn khóa.

Áp dụng:

- **Control bắt buộc = fetch JWKS qua HTTPS/TLS** + pin issuer/CA. Điểm cần siết là `jwt_verifier.py:7`:

```
JWKS_URL = os.getenv("JWKS_URL", "http://keycloak:8080/.../certs") # dev: HTTP nội bộ
```

→ Production đặt `JWKS_URL=https://...` để response JWKS được TLS bảo vệ (MITM không thay được nội dung).

- **Bằng chứng:** chạy `python -m tests.security.sec_jwks_substitution_demo` → `verify_token` **chấp nhận token giả** → chứng minh app-logic bất lực → **PHẢI có TLS**.

Cách đọc demo: kết quả "đòn LQT" là cố ý, để minh chứng giới hạn của tầng ứng dụng — khác pytest bảo mật (PASS = đòn bị chặn).

NGƯỜI 2 — HMAC / M2M

KB2 — Sửa nội dung request M2M (Tampering · MT-INTEG)

(A) Hướng tấn công

Cách đánh & vì sao có lỗ hổng. Luồng máy-gọi-máy (M2M) ký mỗi request bằng HMAC. Kẻ tấn công chặn một request đã ký, **sửa body** (đổi id, số tiền...) nhưng **giữ nguyên chữ ký cũ**, hòng tườn dữ liệu độc. Lỗ hổng xảy ra **nếu** chữ ký không gắn chặt với nội dung body.

Hiểu cách HMAC ký (phần khó — giảng kỹ). Chữ ký KHÔNG ký lên body trực tiếp, mà ký lên một **chuỗi chuẩn hóa (canonical request)** trong đó **có chứa hash của body**. Trong scripts/attack_demo.sh:31-39 (hàm sign_hmac), 3 bước:

```
bh=$(sha256hex "$body")           # (1) băm body -> body_hash :33
canon=$(printf 'POST\n/api/service\n\nhost:...\nx-key-id:...\nx-nonce:...\nx-timestamp:...\n\nhost;x-key-id;x-nonce;x-timestamp\n%s' ... "$bh") # (2) ghép canonical, có body_hash :34-35
sts="HMAC-SHA256\n{ts}\ngateway-internal/v1\n"$(sha256hex "$canon") # (3) string-to-sign :37
sig = HMAC-SHA256(secret, sts)      # ký bằng secret :38
```

→ Vì body_hash nằm trong chuỗi được ký, **đổi 1 byte body là chữ ký phải khác**.

Luồng xử lý của đòn đánh: ký lúc body = {"id":1}, nhưng gửi body = {"id":999} kèm chữ ký cũ → server băm lại body 999 → ra chữ ký kỳ vọng khác → lệch.

Payload (theo scripts/attack_demo.sh:73-74):

```
TS2=$(date +%s); NONCE2=$(uuid4)
post_service "$BODY" '{"id":999}' "$TS2" "$NONCE2" # dòng 74 -> ký vọng 401
# ^ký cho {"id":1} ^gửi {"id":999}
```

Giải thích payload: hàm post_service (attack_demo.sh:40-46) nhận **body-để-ký** (tham số 1) và **body-để-gửi** (tham số 2) **khác nhau**: nó ký chữ ký dựa trên {"id":1} nhưng --data-binary gửi đi {"id":999} → mô phỏng đúng đòn "sửa body giữ chữ ký".

(B) Hướng phòng thủ

Mindset: chữ ký phải phủ lên chính nội dung (đổi body là hỏng chữ ký), và so sánh chữ ký **constant-time** để không rò rỉ qua thời gian.

Code chặn — gateway/crypto/hmac_verifier.py:

1. Hash body nằm trong chuỗi ký — hmac_verifier.py:63 (trong _build_canonical_request):

```
body_hash = hashlib.sha256(body).hexdigest() if body else EMPTY_BODY_HASH
```

2. Server tự tính lại chữ ký rồi so sánh — hmac_verifier.py:122-133:

```
expected = compute_signature(method, path, query, signed_headers_values, body, secret) # 129
if not hmac.compare_digest(expected, signature):      # dòng 132 — constant-time
    raise HMACInvalid("invalid_signature")           # dòng 133
```

Giảng kỹ lưỡng: request tới hmac_auth.py:25 → :29 đọc body raw → :30-38 gọi verify_hmac_request. Trong đó, server dùng **chính body nhận được** để băm lại (hmac_verifier.py:63) và tính expected (:129). Body bị đổi thành 999 → expected khác chữ ký đính kèm (ký cho 1) → compare_digest (:132) thất bại → ném HMACInvalid("invalid_signature") → bắt ở hmac_auth.py:39-47 trả **401**. Kẻ tấn công **không có secret** nên không ký lại được cho body mới. compare_digest so sánh constant-time → không lộ qua đo thời gian.

KB3 — Chối bỏ hành vi (Repudiation · MT-NONREP)

(A) Hướng tấn công

Cách đánh & vì sao có lỗ hổng. Một service M2M gửi request gây hại, sau đó **chối** "tôi không gửi". Nếu hệ thống không lưu vết gắn hành động với danh tính, sẽ **không quy trách nhiệm được**. Lỗ hổng = thiếu **sổ audit**.

Payload (theo scripts/attack_demo.sh:105-107): đây không phải payload phá hệ thống, mà là **tình huống cần đối chất**:

```
post_service '{"id":1}' '{"id":1}' "$TSA" "$(uuid4)" # request hợp lệ (allow) :106
post_service '{"id":1}' '{"id":999}' "$TSB" "$(uuid4)" # request tamper (deny) :107
curl -s "$GATEWAY/audit/recent?limit=10"           # đọc sổ audit :108
```

Giải thích payload: bắn 1 request hợp lệ + 1 request xấu, rồi gọi /audit/recent để **lấy bằng chứng**. Cả hai phải xuất hiện trong sổ kèm danh tính dev-key-01.

(B) Hướng phòng thủ

Mindset: mọi quyết định auth (cho qua HAY từ chối) phải được ghi kèm danh tính, không sửa được; ghi cả việc tốt lẫn việc xấu.

Code chặn — gateway/observability/audit.py:29-59 (record_audit):

```
record = {"ts": ..., "channel": channel, "actor": actor,
         "method": method, "path": path, "decision": decision, "reason": reason} # 38-46
RECENT_AUDIT.append(record) # dòng 48 — buffer RAM (đọc qua API)
audit_logger.info("AUDIT %s", json.dumps(record)) # dòng 50
... open(_AUDIT_FILE, "a").write(line) # dòng 52-58 — append-only ra logs/audit.log
```

Được gọi ở **cả hai nhánh** của middleware HMAC:

- Nhánh từ chối — hmac_auth.py:44-46: record_audit(..., decision="deny", reason=...).
- Nhánh cho qua — hmac_auth.py:53-54: record_audit(..., decision="allow").

Giải kỹ luồng: mỗi lần request M2M qua hmac_auth.py, dù allow hay deny, code ghi một bản ghi chứa actor = X-Key-Id (hmac_auth.py:44/:53) + thời điểm + quyết định. Endpoint gateway/main.py:89-97 (/audit/recent) đọc lại RECENT_AUDIT để trình bày. Service **không thể chối** vì danh tính nó đã nằm trong sổ. (Luồng JWT cũng ghi audit ở auth.py:42-44, :58-60, :69-70.)

KB5 — Làm nghẽn dịch vụ (Denial of Service · MT-AVAIL)

(A) Hướng tấn công

Cách đánh & vì sao có lỗ hổng. Kẻ tấn công bắn **hàng loạt request cùng lúc** làm cạn CPU/băng thông → user thật không vào được. Lỗ hổng xảy ra **nếu** không giới hạn tần suất, hoặc đặt kiểm tồn CPU lên trước khiến rác cũng ngốn tài nguyên.

Payload (theo scripts/attack_demo.sh:98-102):

```
for i in $(seq 1 12); do codes="$codes $(code "$GATEWAY/api/public")"; done # dòng 100
# 12 lần gọi -> 10 cái đầu 200, từ cái 11: 429
```

Giải thích payload: vòng lặp gọi /api/public **12 lần** (vượt ngưỡng 10/phút). Mô phỏng một nguồn bắn dồn dập; kỳ vọng dashboard thấy mã 429 xuất hiện.

(B) Hướng phòng thủ

Mindset: giới hạn tần suất theo nguồn (*per-IP*) + kiểm rẻ trước, dặt sau.

Code chặn — gateway/middleware/rate_limit.py:11-20 + gateway/main.py:65-66:

```
# rate_limit.py:11-15 — đếm theo IP, cửa sổ cố định
limiter = Limiter(key_func=get_remote_address, storage_uri=REDIS_URL, strategy="fixed-window")
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler) # :20
```



```
# main.py:65-66 — ngưỡng từng endpoint
@app.get("/api/public")
@limiter.limit("10/minute")      # quá 10/phút/IP -> RateLimitExceeded -> 429
```

Giải kỹ lưỡng: key_func=get_remote_address (rate_limit.py:13) bấm theo IP → mỗi IP có bộ đếm riêng. Decorator @limiter.limit("10/minute") (main.py:66) đếm số lần gọi; lần thứ 11 trong một phút làm ném RateLimitExceeded, được handler đăng ký ở rate_limit.py:20 chuyển thành **429**. Rate-limit là chốt **[1]** rẻ nhất, đặt trước mọi crypto. (/api/protected, /api/admin giới hạn 5/phút — main.py:71, :78.)

NGƯỜI 3 — HẠ TẦNG / TRUYỀN DẪN

4 kịch bản này phần lớn là **control tầng triển khai** (TLS/mTLS/Vault/cô lập mạng). Một số đã thiết kế nhưng chưa hiện thực trong PoC — trình bày trung thực chính là tư duy "kiến trúc ≠ triển khai".

KB4 — Nghe lén đường truyền (Information Disclosure · MT-CONF)

(A) Hướng tấn công

Cách đánh & vì sao có lỗ hổng. Kẻ tấn công nghe lén giao tiếp user/service ↔ backend, lấy access token ngay tại thời điểm đó để mạo danh. Lỗ hổng xảy ra **nếu** kênh truyền là HTTP trần — header Authorization: Bearer ... đi dạng chữ thường, ai bắt gói cũng đọc.

Payload (phản chứng, dạng curl -v):

```
curl -v http://localhost:8000/api/protected -H "Authorization: Bearer <token>"
# -> trong gói tin, dòng "Authorization: Bearer eyJ..." là PLAINTEXT
```

Giải thích payload: cờ -v in ra header thật gửi đi; trên HTTP, token hiện nguyên văn → chứng minh sniffer đọc được.

(B) Hướng phòng thủ

Mindset: chống nghe lén là việc của tầng truyền dẫn (mã hóa kênh), không phải logic ứng dụng.

Áp dụng (control triển khai):

- **TLS termination ở ingress:** prod đặt reverse-proxy/ingress giữ chứng chỉ → người dùng nói HTTPS → sniffer chỉ thấy rác. Mô tả trong infra/docker-compose.prod.yml (chú thích cổng gateway sau ingress giữ TLS).
- **mTLS cửa 2** Gateway↔Backend: mã hóa cả chặng nội bộ.

- **Demo trình bày:** curl -v http://... (phản chứng token plaintext) → giải thích prod HTTPS thì không lộ. Đây là control **không test bằng pytest**.

Trạng thái PoC: dev chạy HTTP trên localhost (chấp nhận vì chỉ nội bộ máy). Prod bắt buộc TLS — đã ghi trong prod compose.

KB7 — Vòng qua Gateway gọi thẳng Backend (Spoofing · S1-GB)

(A) Hướng tấn công

Cách đánh & vì sao có lỗ hổng. Kẻ tấn công đứng **trong mạng nội bộ** (chiếm 1 container phụ, hoặc qua SSRF) và gọi **thẳng backend, bỏ qua toàn bộ** dây chuyền chốt của Gateway. Lỗ hổng xảy ra **nếu** backend tin "ai trong mạng cũng hợp lệ" (mô hình phẳng) — thủng 1 chỗ là vào thẳng backend.

Payload (khái niệm):

```
# Đứng trong gw-net, gọi thẳng backend, KHÔNG qua gateway:
curl http://backend:8000/internal/...
```

(B) Hướng phòng thủ

Mindset: zero-trust ở tầng mạng — vị trí trong mạng **KHÔNG** phải bằng chứng tin cậy.
Hai lớp:

1. **Cô lập mạng:** backend **không publish** cổng, chỉ tồn tại bằng IP nội bộ. Áp dụng: infra/docker-compose.prod.yml chỉ phơi Gateway, các service khác bỏ ports:.
2. **mTLS cửa 2:** backend chỉ chấp nhận peer **trình chứng chỉ hợp lệ** = Gateway.

Trạng thái PoC (trung thực): PoC chưa có service backend riêng → chưa hiện thực mTLS (GAP thiết kế-vs-triển khai). Lớp cô lập mạng đã demo qua prod compose; mTLS là hạng mục hardening trước prod (cần sinh cert + thêm backend).

KB8 — Backend tin header Gateway chuyển xuống (Elevation · E2-GB)

(A) Hướng tấn công

Cách đánh & vì sao có lỗ hổng. Gateway xác thực xong thường chuyển danh tính/quyền xuống backend qua header (X-User, X-Role). Nếu backend **tin header này vô điều kiện**, kẻ tấn công có token user thường nhưng **tự đặt** X-Role: admin để leo quyền. Lỗ hổng = backend coi header nội bộ là "đã được Gateway đảm bảo" mà không tự kiểm lại.

Payload (theo tests/security/test_backend_zerotrust.py:37-40):

```
user_token = make_token()                # token user THƯỜNG (không admin)
spoof = {"authorization": f"Bearer {user_token}", "x-role": "admin"} # tự nhét X-Role
```

Giải thích payload: token hợp lệ nhưng quyền thấp, kèm header x-role: admin **giả** do client tự đặt — kỳ vọng backend dùng tin header này.

(B) Hướng phòng thủ

Mindset — "chốt 8 zero-trust": backend **KHÔNG** tin header trần. Nó **verify lại token gốc** và đọc quyền từ **claim ĐÃ KÝ**, bỏ qua header do client gửi.

Code chặn (nguyên lý) — tests/security/test_backend_zerotrust.py:19-34:

```
def naive_backend(headers):                # :19-21 SAI: tin header
    return headers.get("x-role") == "admin" # -> bị lừa bởi X-Role giả

def zerotrust_backend(headers):            # :24-34 ĐÚNG: verify lại token
    claims = verify_token(headers["authorization"][7:]) # :31 đọc role từ claim ĐÃ KÝ
    return "admin" in claims.get("realm_access", {}).get("roles", []) # :34
```

Giảng kỹ lưỡng: zerotrust_backend (:31) gọi lại verify_token (chữ ký IdP) trên token gốc, rồi lấy role từ realm_access.roles trong **claim đã ký** (:34) — hoàn toàn **bỏ qua** x-role header. Token user thường → không có admin → trả False (chặn). naive_backend (:21) đọc thẳng x-role → bị lừa. (SEC-14.)

Trạng thái PoC: chưa có backend thật nên đây là **test nguyên lý** (dùng chính verify_token làm bước "backend kiểm lại"). Khi thêm backend phải áp pattern này.

KB10 — Lộ khóa ở kênh Gateway↔Vault (Information Disclosure · I1-Vault)

(A) Hướng tấn công

Cách đánh & vì sao có lỗ hổng. Nếu kênh Gateway↔Vault **không mã hóa** hoặc cấu hình sai (token quá rộng / để root token), ai nghe lén/đứng trong nội bộ đều **đọc được secret/khóa**. Kho bí mật là tài sản quý nhất.

Payload (khái niệm):

```
# Vault dev mở HTTP + root token -> đọc thẳng secret nếu chạm được:
curl -H "X-Vault-Token: dev-root-token" http://vault:8200/v1/secret/data/gateway/hs256
```

(B) Hướng phòng thủ

Mindset: least-privilege + mã hóa kênh + không dùng root token.

Áp dụng:

1. **TLS tới Vault** (chống nghe lén in-transit).
2. **Least-privilege policy**: mỗi service token chỉ đọc đúng path của mình, **bỏ root token**.
3. **Fail-closed ở code (ĐÃ LÀM — F4) — gateway/storage/vault_client.py:17 + :32-36**:

```
VAULT_TOKEN = os.getenv("VAULT_TOKEN")          # :17 — bỏ default "dev-root-token"
def _require_token():                             # :32-36
    if not VAULT_TOKEN:
        raise VaultError("vault_token_not_configured") # thiếu token -> TỪ CHỐI
    return VAULT_TOKEN
```

Giảng kỹ lưỡng: mọi lần đọc Vault (vault_client.py:47, :71) đều gắn header X-Vault-Token: _require_token(). Nếu không cấu hình token, _require_token (:34-35) ném VaultError ngay → Gateway **không gọi Vault** (fail-closed), thay vì dùng root token hardcoded. Còn **TLS + policy least-priv** là control triển khai: prod đặt Vault internal-only (docker-compose.prod.yml) và bật TLS thật.

Trạng thái PoC: dev dùng Vault HTTP + root token (lỗi tắt dev) — đã siết phần code (fail-closed), phần TLS/policy là hardening trước prod.

Bảng tra nhanh toàn bộ (in ra trình bày)

KB	Tên	STRIDE / MT	Người	Payload (.sh / test)	Code chặn (file:line)	Kết quả
1	Token giả / alg=none	Spoofing / AUTHN	1	attack_demo.sh:63-66	jwt_verifier.py:56-57, 99-110	401
6	Leo thang quyền	Elevation / AUTHZ	1	attack_demo.sh:90	auth.py:50-64	403
9	Tráo JWKS	Spoofing IdP / AUTHN	1	sec_jwks_substitution_demo.py	jwt_verifier.py:7 (JWKS-over-TLS)	cần TLS
2	Sửa body HMAC	Tampering / INTEG	2	attack_demo.sh:73-74	hmac_verifier.py:63, 129-133	401
3	Chối bỏ	Repudiation / NONREP	2	attack_demo.sh:105-108	audit.py:29-59, hmac_auth.py:44-54	ghi sổ
5	Flood DoS	DoS / AVAIL	2	attack_demo.sh:100	rate_limit.py:11-20, main.py:65-66	429
4	Nghe lên token	Info Disclosure / CONF	3	curl -v http://...	TLS ingress (docker-compose.prod.yml)	mã hóa kênh
7	Vòng qua gateway	Spoofing / S1-GB	3	curl http://backend...	cô lập mạng + mTLS (thiết kế)	cách ly
8	Backend tin header	Elevation / E2-GB	3	test_backend_zerotrust.py:37-40	test_backend_zerotrust.py:24-34	chặn spoof
10	Lộ khóa Vault	Info Disclosure / I1-Vault	3	curl -H X-Vault-Token...	vault_client.py:17, 32-36	từ chối

Mỗi người trình bày 3–4 kịch bản theo bảng, mỗi cái nói đủ 2 ý: **tấn công** (cách + vì sao + payload có dòng cụ thể) và **phòng thủ** (mindset + code file:line + giảng luồng).